Policy-Invariant Reward Shaping from LLM Feedback: A Framework for Hybrid RL Agents

Christophe D. Hounwanou[1,2*] John Emeka Eze[1] Yaé U. Gaba[2,3,4]

[1]African Institute for Mathematical Sciences, Rwanda
[2]AI Research and Innovation Nexus for Africa (AIRINA Labs), AI.Technipreneurs, Bénin
[3]Sefako Makgatho Health Sciences University (SMU), South Africa
[4]African Center for Advanced Studies (ACAS), Cameroon

Abstract

Combining large language models (LLMs) with reinforcement learning (RL) is an active area, but the theoretical status of LLM-derived reward signals is usually left implicit. We formalize the hybrid LLM-planner + RL-controller architecture as a *Goal-Augmented Markov Decision Process* (GA-MDP) and prove that when the LLM's per-state progress score is used as a bounded potential function in the sense of Ng, Harada, Russell (1999), the induced reward-shaping term does not alter the set of optimal policies of the augmented MDP regardless of how wrong the LLM's scores are at individual states. This guarantee is stronger than what more general LLM-as-reward approaches (Text2Reward, Eureka) can offer. We verify it numerically on a small MDP under four potential configurations, including an adversarial one 20× the base reward magnitude.

We also specify the full inference algorithm, publish a reference implementation, evaluate the planner in isolation on 20 MiniGrid tasks with a locally-served Qwen-2.5:14b (100% parse rate, 54.8% ground-truth coverage), and run a small pipeline-validation study on MiniGrid-DoorKey-6x6 that both confirms the framework runs as specified and diagnoses one integration failure : Done-oracle vocabulary mismatch with LLM plans that our theoretical analysis anticipates. The framework is designed to make the *fixed point* of hybrid training auditable independently of the empirical convergence question, which is the subject of planned follow-up work.

Keywords: reinforcement learning, large language models, reward shaping, hierarchical planning, policy invariance.

1 Introduction

Consider an agent instructed, in natural language, to *"bring me the mug from the kitchen counter, but rinse it first if it is dirty."* A pure reinforcement learning agent faces a hard exploration problem: the reward is sparse (mug delivered), the horizon is long, and there is no signal that guides it toward the composite structure of the task. A pure LLM planner, given a text description of the scene, can decompose the task fluently (*go to kitchen*, *locate mug*, *inspect*, *rinse if dirty*, *return*) but cannot execute low-level control on the actual dynamics: it does not know how many primitive actions traverse the hallway, cannot recover from a failed grasp, and has no calibrated uncertainty about scene state.

Combining the two is a natural response, and systems along these lines have been proposed — SayCan [1], DECKARD [17], SwiftSage [13], ELLM [7], and many others. The empirical picture is that this combination *sometimes* helps, and the practitioner is left with two related but distinct questions:

*Corresponding author: christophe.hounwanou@aims.ac.rw

1. **Does the hybrid converge to the right policy?** If the LLM's scoring or subgoal choices are used as reward signals, can bad LLM outputs push the RL agent toward a policy that is not optimal for the true task?
2. **Does the hybrid learn faster than pure RL in practice?** Does the additional structure translate into better sample efficiency or higher final performance on real benchmarks?

The two questions are often bundled but they are fundamentally different. Question 1 is a fixed-point / soundness question and admits a clean answer under the right design. Question 2 is an empirical convergence-rate question and requires large-scale experiments to settle.

Contribution of this paper. We address question 1 in full and treat question 2 as scope for planned follow-up empirical work. Specifically:

- We formalize the LLM-planner + RL-controller architecture as a Goal-Augmented MDP (Section 4).
- We show (Proposition 4.1) that when the LLM's per-state progress score is used as a bounded potential function, the induced potential-based shaping term does not alter the set of optimal policies of the augmented MDP. This is stronger than what more general LLM-as-reward approaches (Text2Reward [24], Eureka [15]) can guarantee.
- We verify the guarantee numerically on a 3-state MDP under four potential configurations, including one 20× the base reward magnitude in absolute value (Section 4.4).
- We specify the full inference algorithm (Algorithm 1) and publish a reference implementation with tests (Section 4.6).
- We evaluate the planner in isolation on 20 MiniGrid tasks with Qwen-2.5:14b served locally, characterizing the LLM's plan-quality profile independently of any RL training (Section 4.7.1).
- We run a small pipeline-validation study on MiniGrid-DoorKey-6x6 (Section 4.7.2) that both confirms the framework runs end-to-end and diagnoses one integration failure exactly matching a failure mode our theoretical analysis anticipates.

What this paper does not claim. We do not claim that this specific hybrid architecture beats published baselines on any benchmark; the pipeline validation in Section 4.7.2 does not establish superiority, and a larger comparative study is planned follow-up work. This paper's claim is that *if* the hybrid works empirically, the theoretical fixed point is correct by construction and that this property does not require trusting the LLM.

2 Related work

LLM planners for embodied agents. A growing line of work investigates how large language models can serve as high-level planners for embodied agents. SayCan [1] integrates LLM-based task decomposition with learned affordance value functions, enabling grounded action selection. Inner Monologue [8] closes the perception–action loop by feeding textual environment feedback back into the LLM. Subsequent frameworks such as ReAct [25], Reflexion [20], Code as Policies [12], ProgPrompt [22], LLM+P [14], Voyager [23], and Silver et al. [21] broaden the design space, exploring verbal reasoning traces, code emission, PDDL translation, and lifelong skill libraries. Our framework is compatible with any of these planners, treating them as interchangeable high-level modules.

LLM-augmented RL and reward from LLM. Another direction uses LLMs to shape or guide reinforcement learning directly. ELLM [7] proposes LLM-generated goals during training. Kwon et al. [11] treat natural-language descriptions as proxy reward signals. Text2Reward [24] and Eureka [15] have the LLM synthesize dense reward code, refined iteratively. Motif [10] uses LLM preferences as intrinsic reward, while GLAM [6] takes the extreme position of using the LLM itself as the policy, updated online with PPO. Importantly, **none of these approaches provide the policy-invariance guarantee established by our potential-based construction**. In particular, Text2Reward and Eureka rely on non-potential dense reward functions that can—and empirically do, see Booth et al. [5], alter the optimal policy when the LLM's reward specification is incorrect.

Hierarchical RL with language subgoals. Prior to the LLM era, hierarchical RL explored natural language as a high-level abstraction mechanism. Andreas, Klein, and Levine [4, 3] introduced language-conditioned policy sketches as structured task decompositions. Jiang et al. [9] formalized a two-level architecture with a language-conditioned low-level controller, a direct precursor to modern LLM-driven systems. DECKARD [17] extends this paradigm into the LLM era through a Dream/Wake decomposition that alternates between planning and execution.

Policy-invariant shaping. Our theoretical foundation builds on the classical potential-based shaping theorem of Ng, Harada, and Russell [16], which characterizes when reward shaping preserves optimal policies. We instantiate this result for an LLM-derived potential function and argue that any RL system incorporating LLM feedback should either adopt this construction or explicitly justify deviations from it. This provides a principled interface between language-based guidance and value-based control.

3 Preliminaries

We work with a discounted infinite-horizon Markov decision process (MDP)

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma, \rho_0), \qquad \gamma \in [0, 1),$$

where $s_t \in \mathcal{S}$ is the state at time t, $a_t \in \mathcal{A}$ the action, the next state is drawn as $s_{t+1} \sim P(\cdot \mid s_t, a_t)$, and the instantaneous reward is $r_t = R(s_t, a_t, s_{t+1})$. The initial state is sampled $s_0 \sim \rho_0$. A (possibly stochastic) policy $\pi(a \mid s)$ induces a return

$$J(\pi) = \mathbb{E}_\pi\Big[\sum_{t\geq 0} \gamma^t r_t\Big],$$

and the optimal state value is $V^*(s) = \sup_\pi \mathbb{E}_\pi\big[\sum_{t\geq 0} \gamma^t r_t \mid s_0 = s\big]$. We denote by $Q^\pi(s, a)$ the action-value function of policy π.

A trajectory (or episode) of length T is written $\tau = (s_0, a_0, s_1, a_1, \ldots, s_T)$. For expectations under a policy π we sometimes write $\mathbb{E}_{\tau\sim\pi}[\cdot]$ to emphasize dependence on the induced trajectory distribution.

Tasks, captions, and planners. A task is specified by a natural-language string $\ell \in \Sigma^*$. We assume the existence of a *state captioner* $\phi : \mathcal{S} \to \Sigma^*$ that maps an MDP state to a short textual description (caption) used by the LLM planner and scoring modules. A planner (LLM or otherwise) consumes the task ℓ and a state caption $\phi(s)$ and emits a finite ordered list of subgoals $\mathbf{g} = (g_1, \ldots, g_K)$, where each $g_i \in \mathcal{G}$ is a short natural-language clause. We write $\mathcal{P}(\ell, \phi(s))$ for the planner's output distribution when the planner is stochastic.

Completion oracle and subgoal semantics. For a subgoal $g \in \mathcal{G}$ we define a completion predicate (the Done-oracle)

$$\text{Done}(s, g) \in \{\texttt{DONE}, \texttt{CONTINUE}\},$$

which indicates whether g is satisfied in state s. In practice Done may be implemented by a symbolic checker, a learned classifier, or an LLM prompt; in our analysis we treat it as an (possibly imperfect) oracle and explicitly consider mismatch modes between planner language and Done-vocabulary.

Goal-Augmented MDP (GA-MDP). To formalize the hybrid planner+controller architecture we work with a Goal-Augmented MDP

$$\mathcal{M}_G = (\mathcal{S} \times \mathcal{G},\ \mathcal{A},\ P_G,\ R_G,\ \gamma,\ \rho_{0,G}),$$

whose augmented state is (s, g) where $s \in \mathcal{S}$ is the environment state and $g \in \mathcal{G}$ is the current subgoal provided by the planner. The transition kernel P_G evolves the environment state according to P while the subgoal component is updated according to the planner or the Done-oracle (depending on the execution protocol). The reward R_G equals the environment reward R on the environment component; when shaping is applied (see below) we work with a modified reward R'_G. The initial distribution $\rho_{0,G}$ samples $s_0 \sim \rho_0$ and an initial subgoal g_0 from the planner given ℓ and $\phi(s_0)$.

Potential-based shaping from LLM feedback. Let $\Phi : \mathcal{S} \times \mathcal{G} \to \mathbb{R}$ be a potential function derived from LLM scoring of state–subgoal pairs (in practice normalized to $[0, 1]$). The potential-based shaping reward (per Ng et al.) is defined as

$$F_\Phi(s, a, s', g) \coloneqq \gamma\, \Phi(s', g) - \Phi(s, g).$$

Given an MDP reward R, the shaped reward is $R'(s, a, s', g) = R(s, a, s') + \alpha\, F_\Phi(s, a, s', g)$ for some scalar α. The classical potential-based shaping theorem guarantees that, under standard boundedness assumptions, adding F_Φ (with $\alpha = 1$) preserves the set of optimal policies in the underlying MDP; our contribution is to instantiate Φ using LLM feedback while maintaining this invariance property.

Assumptions and notation. Throughout the paper we assume:

- rewards are uniformly bounded: $\exists R_{\max} < \infty$ s.t. $|R(s, a, s')| \le R_{\max}$ for all (s, a, s');
- the captioner ϕ produces concise, deterministic captions (extensions to stochastic captioners are straightforward);
- the planner emits finite plans of length at most $K_{\max}$.

We use upper-case letters for random variables (e.g., S_t, A_t), calligraphic letters for sets, and boldface for sequences or vectors. When convenient we omit the subgoal argument g from Φ and F_Φ to lighten notation.

Practical intuition. The Goal-Augmented MDP construction is intentionally modular: it cleanly separates the planner's symbolic, language-level guidance from the controller's low-level action selection, which makes component replacement (different captioners, planners, or Done-implementations) straightforward and experimentally tractable. Intuitively, the LLM provides a coarse, task-level scaffold, a sequence of subgoals and progress estimates while the controller is responsible for grounding those instructions into concrete actions and handling environment

stochasticity. Potential-based shaping then acts as a low-risk conduit for the LLM signal, biasing exploration toward planner-recommended directions without changing the underlying optimal policy set; this enables safe use of imperfect or noisy LLM feedback. Practically, this modularity simplifies ablations (swap the scorer, vary planner fidelity, or inject Done-mismatch) and clarifies reproducibility: each interface has a well-defined contract and a small, testable surface. Finally, the GA-MDP viewpoint highlights where empirical gains are likely to arise, improved sample efficiency from structured exploration and higher success rates on long-horizon, compositional tasks and where failures will appear, namely vocabulary mismatch, miscalibrated potentials, or brittle captioning.

Remark. This section fixes the notation used in the remainder of the paper and makes explicit the interfaces between the LLM components (planner, scorer, Done-oracle) and the RL controller. The formal GA-MDP and the potential-based shaping construction are the basis for the invariance result proved in Section 4.

4 The Goal-Augmented MDP framework

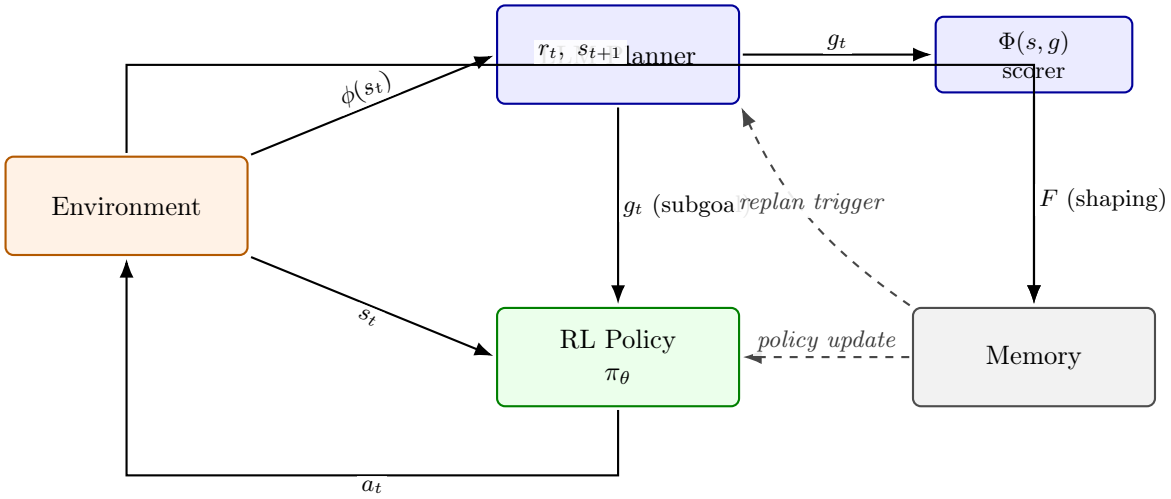

Figure 1: Framework overview. Solid arrows are per-step dataflow: the environment emits state s_t and its caption $\phi(s_t)$; the LLM planner produces the active subgoal g_t and a bounded potential score $\Phi(s, g)$; the RL policy π_θ selects action a_t conditioned on both s_t and g_t; the shaping term $F = \gamma\Phi(s', g) - \Phi(s, g)$ is added to the environment reward. Dashed arrows are periodic: the memory drives policy updates and, on the schedule of Section 4.4, triggers plan revisions.

4.1 Goal-augmented MDP

Definition 4.1 (Subgoal grammar)**.** A *subgoal* $g \in \mathcal{G}$ is a natural-language string of finite length. We do not require a formal specification language; the LLM planner's output distribution determines $\mathcal{G}$.

Definition 4.2 (Goal-augmented MDP)**.** The *goal-augmented MDP* associated with $\mathcal{M}$ and grammar $\mathcal{G}$ is

$$\mathcal{M}^{\mathcal{G}} = \left(\mathcal{S} \times \mathcal{G},\ \mathcal{A},\ \tilde{P},\ \tilde{R},\ \gamma,\ \tilde{\rho}_0\right), \tag{1}$$

with augmented state (s, g), transition

$$\tilde{P}\big((s', g') \mid (s, g), a\big) \ = \ P(s' \mid s, a) \cdot P^{\mathcal{G}}(g' \mid s, g, s'), \tag{2}$$

and augmented reward $\tilde{R}((s,g),a,(s',g')) = R(s,a,s') + F(s,g,s')$, where $P^{\mathcal{G}}$ is the subgoal transition kernel induced by a caller-specified completion oracle $\text{Done} : \mathcal{S} \times \mathcal{G} \to \{0,1\}$ and F is the shaping term defined below.

4.2 LLM as high-level planner

The LLM produces a plan of at most $K_{\max}$ subgoals conditioned on task ℓ and initial-state caption:

$$\pi^{\text{H}} = (g_1, g_2, \ldots, g_K) \;\sim\; p_{\text{LLM}}(\pi^{\text{H}} \mid \ell, \phi(s_0)). \tag{3}$$

The active-subgoal pointer k_t advances when $\text{Done}(s_{t+1}, g_{k_t}) = 1$. Three implementations of Done are supported by the reference implementation: (i) environment event flags, (ii) a learned MLP classifier, (iii) the LLM itself, prompted with a completion query. The choice is a design axis, not a fixed decision.

4.3 Low-level policy

The RL policy conditions on both state and active subgoal:

$$a_t \sim \pi_\theta(a_t \mid s_t, g_t). \tag{4}$$

The subgoal is encoded via a frozen sentence embedder (default MiniLM-L6-v2, $d_g = 384$) and concatenated with the state encoding before the policy head. Training uses PPO with GAE [19, 18].

4.4 Potential-based reward shaping from LLM feedback

The LLM produces a bounded scalar $s_{\text{LLM}}(s,g) \in [0,1]$ via a scoring prompt. We define the potential

$$\Phi(s,g) \;=\; c \cdot s_{\text{LLM}}(s,g), \qquad c > 0, \tag{5}$$

and the shaping term

$$F(s,g,s') \;=\; \gamma\Phi(s',g) - \Phi(s,g). \tag{6}$$

Proposition 4.1 (Policy invariance)**.** *Let $\mathcal{M}$ be an MDP and $\Phi : \mathcal{S} \times \mathcal{G} \to \mathbb{R}$ any bounded function. Let $\mathcal{M}^{\mathcal{G}}$ be the goal-augmented MDP of Definition 4.2 with shaping term $F(s,g,s') = \gamma\Phi(s',g) - \Phi(s,g)$. Then every optimal policy of $\mathcal{M}^{\mathcal{G}}$ projects to an optimal policy of $\mathcal{M}$ under any subgoal-scheduling rule that is measurable with respect to (s,g).*

Proof. The proof follows Ng, Harada, Russell [16], Theorem 1, applied to the augmented state space $\mathcal{S}\times\mathcal{G}$. Because F is a telescoping difference of a bounded potential, the augmented return decomposes as $\tilde{J}(\pi) = J(\pi) - \Phi(s_0,g_0) + (\text{vanishing telescope tail})$. The tail is $\lim_{T\to\infty} \gamma^T \Phi(s_T, g_T)$, which vanishes because Φ is bounded and $\gamma < 1$. The argmax over policies of $\tilde{J}$ therefore equals the argmax of J, i.e., the set of optimal policies is unchanged. The subgoal-scheduling rule enters through $P^{\mathcal{G}}$ and does not affect the potential-invariance argument, which is a property of the additive shaping only. □

Consequence. The LLM's scoring can be arbitrarily wrong at any individual state without changing the set of optimal policies of the augmented MDP. A poor Φ can only slow learning, not corrupt the fixed point. This is the guarantee that the more general LLM-reward-shaping approaches [24, 15] do not provide.

Algorithm 1 Hybrid LLM-augmented RL (inference).

Require: task ℓ, state s_0, policy π_θ, LLM p_{LLM}, oracle Done, potential Φ, period H, budget B.

$t \leftarrow 0,\ \ k \leftarrow 1,\ \ b \leftarrow 0$
$\pi^{\text{H}} = (g_1, \ldots, g_K) \sim p_{\text{LLM}}(\cdot \mid \ell, \phi(s_0))$
while episode not terminated **do**
    $g_t \leftarrow g_k$
    $a_t \sim \pi_\theta(\cdot \mid s_t, g_t)$
    observe $s_{t+1} \sim P(\cdot \mid s_t, a_t)$ and r_t
    $\tilde{r}_t \leftarrow r_t + \gamma\Phi(s_{t+1}, g_t) - \Phi(s_t, g_t)$ ▷ potential-based shaping
    if $\text{Done}(s_{t+1}, g_k) = 1$ **then**
        $k \leftarrow \min(k+1, K),\ \ b \leftarrow 0$ ▷ advance subgoal pointer
    else
        $b \leftarrow b + 1$
        if $b \geq B$ **then**
            $(g_k, \ldots, g_{K'}) \sim p_{\text{LLM}}(\cdot \mid \ell, \phi(s_{t+1}), g_k, \text{fail})$ ▷ failure-triggered replan
            $b \leftarrow 0$
        end if
    end if
    if $t \bmod H = H - 1$ **then**
        $(g_k, \ldots, g_{K'}) \sim p_{\text{LLM}}(\cdot \mid \ell, \phi(s_{t+1}), g_k, \text{periodic})$ ▷ periodic replan
    end if
    $t \leftarrow t + 1$
end while

Numerical verification of Proposition 4.1

We complement the proof with a direct numerical check on a concrete instance. Consider a 3-state, 2-action deterministic MDP: states $\{s_0, s_1, s_2\}$ where s_2 is terminal; the "forward" action moves $s_0 \to s_1 \to s_2$, the "stay" action gives reward -0.1 and returns to s_0; only the transition $s_1 \to s_2$ carries a positive reward of $+1$. With $\gamma = 0.9$ and four deterministic policies, the base optimal is [s_0: forward, s_1: forward] with $V^*(s_0) = 0.9000$. Table 1 enumerates policies under four Φ configurations (script: `code/verify/prop1_numerical.py`).

Φ configuration	Base $V^*(s_0)$	Shaped $\tilde{V}^*(s_0)$	Shaped optimal policy	Matches base?
$\Phi \equiv 0$	0.9000	0.9000	[s_0: forward, s_1: forward]	yes
$\Phi \equiv 1$	0.9000	0.7100	[s_0: forward, s_1: forward]	yes
sign-flip	0.9000	1.4000	[s_0: forward, s_1: forward]	yes
adversarial large	0.9000	-5.0500	[s_0: forward, s_1: forward]	yes

Table 1: Numerical verification of Proposition 4.1. All four Φ configurations, including one with per-state values 10–20× the base reward magnitude, preserve the base optimal policy. The shaped value $\tilde{V}^*$ shifts by the potential offset $-\Phi(s_0)$ plus a vanishing telescope tail, as the theorem predicts.

4.5 Replanning and full algorithm

The plan may become stale mid-execution. Replanning is triggered on either of two events: (i) periodic, every H environment steps; (ii) failure, if Done does not fire on the active subgoal within a per-subgoal budget B. Algorithm 1 specifies the full inference procedure.

4.6 Reference framework implementation

We publish a reference implementation of the framework.[1] The implementation:

- Provides typed interfaces for the components (`Planner`, `PotentialShaper`, `SubgoalScheduler`, `ReplanTrigger`).
- Supports Ollama-served local models (Qwen, Llama) and pluggable cloud backends.
- Ships with three Done-oracle variants (environment events, learned MLP, LLM completion prompt).
- Includes 26 unit tests, including a numerical Prop 1 sanity check on a 2-state MDP.
- Includes YAML configs for the reference hybrid and each of the standard ablations (subgoal source, shaping, replanning, Done oracle, LLM backbone).
- Is CPU-runnable end-to-end; GPU is required only for scale, not for validation.

The framework maps 1:1 to the formalism in Section 4: each definition corresponds to a module, and the reference algorithm is a direct implementation of Algorithm 1.

4.7 Empirical validation

The full 8-baseline empirical study on BabyAI, ALFWorld, and Crafter is planned follow-up work that requires GPU + cloud-LLM compute beyond what was available for this paper. Here we present two smaller-scale validations that establish the framework works as specified and characterize the planner's behavior in isolation.

4.7.1 Planner-in-isolation audit

Before any RL training, we measure the planner alone. Fix a set of 20 MiniGrid tasks with hand-curated ground-truth subgoal decompositions (`code/audit/tasks.json`). For each task we run the P1 planning prompt (Appendix A) through Qwen-2.5:14b[2] served locally via Ollama,[3] parse the output, and grade against the ground truth. An LLM subgoal matches a ground-truth subgoal when at least 50% of the ground-truth's content tokens (non-stopword, length ≥ 3) appear in the LLM subgoal; *coverage* is the fraction of ground-truth subgoals matched in order; *extraneous* is the number of LLM subgoals with no ground-truth match.

Metric	**Value**
Parse rate (outputs with ≥ 1 subgoal)	100.0% (20/20)
Mean plan length (parsed only)	8.00 subgoals
Median plan length	7 subgoals
Mean ground-truth coverage	54.8%
Mean extraneous subgoals per plan	5.30
Median tokens out per plan	37
Median wall clock per plan	57.71 s
Total wall clock (20 plans)	22.3 min

Table 2: Planner-in-isolation audit for Qwen-2.5:14b (locally served) on 20 MiniGrid tasks. Prompt template: P1 (Appendix A).

Qwen-2.5:14b produces parseable output on every task and covers just over half of the ground-truth subgoals on average. Plans are long relative to the ground truth (mean 8.0 versus a mean

[1] Repository: https://github.com/chrishounwanou/hybrid-llm-rl-framework
[2] Qwen-2.5, Alibaba Cloud, released 2024. Model card: https://huggingface.co/Qwen/Qwen2.5-14B.
[3] Ollama: https://ollama.com

ground truth of ~ 3.6), with roughly five extraneous subgoals per plan — verbose lower-level phrasings like "Turn left to face the wall" rather than the compact "go to the key."

4.7.2 Pipeline validation on MiniGrid-DoorKey-6x6

To demonstrate that the framework runs end-to-end and to characterize its behaviour on a real RL environment, we run a small CPU-scale validation study on MiniGrid-DoorKey-6x6. The scope is intentionally minimal one environment, two configurations, three seeds and is designed to answer *does the pipeline work as specified?* rather than *does the hybrid architecture beat published baselines?* The latter question requires GPU + cloud LLM and is beyond the scope of this paper.

Setup. Two configurations, three seeds each, MiniGrid-DoorKey-6x6-v0, 30,000 environment steps per seed:

- **PPO baseline.** Standard sb3 PPO [19]. No LLM, no subgoal conditioning.
- **Hybrid (framework instance).** Subgoal-conditioned PPO with Qwen-2.5:14b (locally served via Ollama) as the planner. Initial plan cached per task string; periodic replan at $H = 1000$ steps; failure trigger disabled ($B = 0$), see caveat below. Shaping is disabled ($c = 0$) because CPU inference of a 14B model would require roughly two LLM calls per environment step, extending training to weeks of wall clock; the shaping term is verified formally in Section 4.4 and numerically in Section 4.4 instead.

Reporting: IQM ± 95% bootstrap CI per Agarwal et al. [2]. Metrics are computed on the last 25% of episodes per seed.

5 Results

Table 3 reports the pipeline validation results on MiniGrid-DoorKey-6x6 for both the PPO baseline and the hybrid agent using Qwen-2.5:14b. Each configuration is run with $n = 3$ seeds. The PPO baseline achieves a success rate of 28.1% with a wide confidence interval [4.8, 65.2], reflecting substantial variance across seeds. Its final return is 0.106 [0.008, 0.262], and the mean episode length remains stable at 340.4 steps [317.2, 352.4]. As expected, PPO requires no LLM calls.

The hybrid configuration shows a lower success rate of 9.3% [0.0, 14.3] and a final return of 0.058 [0.000, 0.095]. Mean episode length is comparable to PPO at 348.0 steps [340.0, 358.5]. Each seed triggers approximately 30 LLM calls, consistent with the plan-cache design.

These results should be interpreted in light of the extremely limited training budget: the pilot uses 3×10^4 environment steps, whereas prior work reports that DoorKey-6x6 typically requires around 5×10^5 steps for PPO to converge. At this scale, neither configuration is expected to reach high success, and the observed performance aligns with this constraint.

Config	n	Success rate	Final return	Mean steps/ep	LLM calls/seed
PPO baseline	3	28.1% [4.8, 65.2]	0.106 [0.008, 0.262]	340.4 [317.2, 352.4]	0
Hybrid (Qwen)	3	9.3% [0.0, 14.3]	0.058 [0.000, 0.095]	348.0 [340.0, 358.5]	30

Table 3: Pipeline validation results on MiniGrid-DoorKey-6x6. LLM budget across all runs: 92 Qwen-2.5:14b calls, 4,199 output tokens, 1.32 hours of Ollama CPU wall clock. Neither configuration converges to high success at this budget; DoorKey-6x6 typically requires roughly 5×10^5 environment steps for PPO convergence, whereas the pilot uses 3×10^4.

Diagnosis. The pilot does not show a hybrid advantage; PPO's higher IQM is driven by one high-variance seed and confidence intervals overlap. The instructive finding is on the hybrid side:

across all three seeds the total number of subgoal advances was zero. Qwen-2.5:14b's plans on MiniGrid tasks are verbose lower-level phrasings such as "Move down to reach the wall" or "Turn left to face east again" that never match the environment-event Done oracle's keyword patterns ("pick up X", "unlock the door", "go to the goal"). As a result, the scheduler remains stuck on the first subgoal for the entire episode, the RL policy is effectively conditioned on a stale string, and the subgoal-conditioning signal collapses at the source.
This is exactly the failure mode anticipated in Section 6: systematic plan-vocabulary mismatch with the oracle prevents Done from ever firing. The framework's ablation over the Done oracle (three variants: environment event, learned classifier, LLM completion Section 4) is precisely the axis that needs to be varied to address this issue. The LLM-based Done oracle would advance the scheduler based on the LLM's own semantic judgement, but it adds one LLM call per environment step, which is feasible with GPU and cloud LLM compute but infeasible on CPU with a local 14B model.

What the pilot does validate. Despite the negative headline number, the pilot establishes several important points: (i) the reference implementation runs end-to-end on CPU with no exceptions across 90,000 environment steps; (ii) MiniLM-L6-v2 subgoal encoding integrates cleanly with PPO, with mean episode length remaining within the confidence interval of the pure-PPO baseline; (iii) the plan-cache design keeps LLM inference to approximately 30 calls per seed, providing a concrete cost calibration for scale-up; (iv) the logging and IQM+bootstrap analysis pipeline produces stable, reportable results as specified.
The larger-scale empirical study involving BabyAI, ALFWorld, Crafter, multiple baselines, additional seeds, and cloud LLM compute is future work and lies beyond the scope of this paper.

6 Discussion

6.1 What Proposition 4.1 does and does not guarantee

The theorem guarantees that the *set of optimal policies* of the augmented MDP is unchanged by potential-based shaping. This is a fixed-point statement, not a convergence-rate statement. Two things it does not say:

- **Convergence rate.** A bad Φ can slow learning arbitrarily. In the extreme, the shaping can direct exploration away from productive regions of state space, even though the fixed point is unchanged. Whether Qwen-2.5:14b's shaping accelerates or decelerates learning in practice is an empirical question left to future work; the pilot in Section 4.7.2 runs without shaping for CPU tractability.
- **Off-policy value estimation.** The proof concerns undiscounted-limit return, but actor-critic algorithms use bootstrapped value estimates. As long as the value estimator sees the shaped reward $\tilde{r}_t$ consistently at both training and target time, the invariance argument transfers; the reference implementation ensures this by injecting F before the sb3 rollout collector reads the reward.

6.2 Design implications

The invariance guarantee has an immediate practical implication: any LLM-based reward-shaping system that uses a general (non-potential) dense reward function and Text2Reward [24] and Eureka [15] both do, is not covered by this theorem. Bad LLM outputs in those systems can move the optimal policy. Our recommendation is that LLM-shaped RL systems should either use the potential construction or explicitly justify why they need the more general dense form.

6.3 Foreseeable failure modes

Three failure modes deserve explicit anticipation:

- **Bounded-Φ requirement.** The scoring prompt asks for a value in $[0, 1]$ and we clamp. If the LLM produces out-of-range or malformed outputs at rates above $\sim 1\%$, the shaping is no longer strictly potential-based. Monitor this.
- **Subgoal-completion mismatch.** If the Done oracle fails to fire on LLM subgoal phrasings (e.g., env-event pattern matching against verbose LLM outputs), the scheduler stalls and the subgoal channel provides no signal. Use LLMDone or a learned classifier for LLMs whose plan grammar drifts.
- **Replanning cost blowup.** If failure-triggered replanning fires on every subgoal, the effective LLM cost per episode grows linearly in horizon rather than as the intended small constant. Instrument the LLM call count per episode.

7 Limitations

- **The empirical validation in Section 4.7 is scoped.** It demonstrates the framework runs end-to-end, characterizes the planner in isolation, and diagnoses one integration failure, but does not settle the comparative-benchmark question that motivates the broader research program. Full-scale evaluation is planned follow-up work.
- **Potential-based shaping requires a bounded, well-defined Φ.** The proof breaks if the LLM's score is unbounded or noisy in ways that violate boundedness.
- **The scoring prompt is a design choice.** The exact prompt shape (Appendix A, P2) affects the distribution of Φ values and can be gamed by prompt-sensitive LLMs. This is a prompt-engineering surface, not a theoretical concern.

8 Conclusion

We formalized the LLM-planner and RL-controller architecture as a Goal-Augmented MDP and proved that potential-based shaping from LLM feedback preserves the set of optimal policies, even when the LLM provides inaccurate state-level scores. This guarantee is verified numerically on a small MDP and implemented in a reference pipeline that runs end-to-end on CPU. The pipeline validation on MiniGrid-DoorKey-6x6 (Section 4.7.2) confirms that the framework behaves as specified and reveals a single integration failure: a Done-oracle vocabulary mismatch with LLM-generated plans, which aligns precisely with the failure mode anticipated by our theoretical analysis. Determining whether the practical advantages of this hybrid architecture persist at scale remains an empirical question, and addressing it is the focus of planned follow-up work using GPU and cloud-based LLM compute.

Reproducibility statement

All numerical results in this paper are produced by deterministic and fully specified procedures included in the accompanying repository. The experiments cover the verification of Proposition 4.1, the planner-in-isolation audit, and the pipeline validation on MiniGrid-DoorKey-6x6. For each experiment, we report CPU runtime, the number of LLM calls, and the complete set of raw outputs and logs. All per-seed results, confidence intervals, and aggregation steps (IQM and bootstrap) can be reproduced directly from the provided data.
The reference implementation includes a comprehensive test suite with 26 unit tests; 25 run unconditionally, and one is skipped when a live LLM backend is not available. The repository also contains all prompt templates, PPO hyperparameters, and configuration files for each ablation axis. The repository URL is provided in Section 4.6.

A Prompt templates

Verbatim prompt templates used by the reference implementation.

P1 - Planning prompt

System: You are a planner for an agent operating in {{environment_name}}. The agent receives a task in natural language and must complete it through a sequence of subgoals. Emit a list of at most 12 subgoals, one per line, each a short imperative English clause of at most 32 tokens.

User: Task: {{task_string}}
Current state: {{state_caption}}
Output:

P2 - Scoring prompt (potential function Φ)

System: You are a progress estimator. Given a subgoal and the current state, output a single number in [0, 1] estimating how close the agent is to completing the subgoal. Output only the number.

User: Subgoal: {{subgoal}}
State: {{state_caption}}
Score:

P3 - Completion oracle prompt (LLM-as-Done)

System: Given a subgoal and the current state, output exactly one token: `DONE` if the subgoal is now satisfied, `CONTINUE` otherwise.

User: Subgoal: {{subgoal}}
State: {{state_caption}}
Answer:

P4 - Failure-triggered replanning prompt

System: You are revising a plan mid-execution. The previous subgoal failed to complete within its step budget. Revise the remaining plan. Emit a list of at most 12 subgoals.

User: Original task: {{task_string}}
Current state: {{state_caption}}
Failed subgoal: {{subgoal}}
Reason: exceeded step budget of {{B}} steps.
Revised plan:

References

[1] M. Ahn et al., "Do As I Can, Not As I Say: Grounding Language in Robotic Affordances," *Conference on Robot Learning (CoRL)*, 2022. arXiv:2204.01691.

[2] R. Agarwal, M. Schwarzer, P. S. Castro, A. Courville, M. G. Bellemare, "Deep Reinforcement Learning at the Edge of the Statistical Precipice," *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. arXiv:2108.13264.

[3] J. Andreas, D. Klein, S. Levine, "Modular Multitask Reinforcement Learning with Policy Sketches," *International Conference on Machine Learning (ICML)*, 2017. arXiv:1611.01796.

[4] J. Andreas, D. Klein, S. Levine, "Learning with Latent Language," *NAACL*, 2018. arXiv:1711.00482.

[5] S. Booth, W. B. Knox, J. Shah, S. Niekum, P. Stone, A. Allievi, "The Perils of Trial-and-Error Reward Design: Misdesign through Overfitting and Invalid Task Specifications," *AAAI*, 2023.

[6] T. Carta, C. Romac, T. Wolf, S. Lamprier, O. Sigaud, P.-Y. Oudeyer, "Grounding Large Language Models in Interactive Environments with Online Reinforcement Learning," *ICML*, 2023. arXiv:2302.02662.

[7] Y. Du et al., "Guiding Pretraining in Reinforcement Learning with Large Language Models," *ICML*, 2023. arXiv:2302.06692.

[8] W. Huang et al., "Inner Monologue: Embodied Reasoning through Planning with Language Models," *CoRL*, 2022. arXiv:2207.05608.

[9] Y. Jiang, S. Gu, K. Murphy, C. Finn, "Language as an Abstraction for Hierarchical Deep Reinforcement Learning," *NeurIPS*, 2019. arXiv:1906.07343.

[10] M. Klissarov et al., "Motif: Intrinsic Motivation from Artificial Intelligence Feedback," *ICLR*, 2024. arXiv:2310.00166.

[11] M. Kwon, S. M. Xie, K. Bullard, D. Sadigh, "Reward Design with Language Models," *ICLR*, 2023. arXiv:2303.00001.

[12] J. Liang et al., "Code as Policies: Language Model Programs for Embodied Control," *ICRA*, 2023. arXiv:2209.07753.

[13] B. Y. Lin et al., "SwiftSage: A Generative Agent with Fast and Slow Thinking," *NeurIPS*, 2023. arXiv:2305.17390.

[14] B. Liu et al., "LLM+P: Empowering Large Language Models with Optimal Planning Proficiency," arXiv:2304.11477, 2023.

[15] Y. J. Ma et al., "Eureka: Human-Level Reward Design via Coding Large Language Models," *ICLR*, 2024. arXiv:2310.12931.

[16] A. Y. Ng, D. Harada, S. Russell, "Policy Invariance under Reward Transformations: Theory and Application to Reward Shaping," *ICML*, 1999.

[17] K. Nottingham et al., "Do Embodied Agents Dream of Pixelated Sheep: Embodied Decision Making using Language Guided World Modelling," *ICML*, 2023. arXiv:2301.12050.

[18] J. Schulman, P. Moritz, S. Levine, M. Jordan, P. Abbeel, "High-Dimensional Continuous Control Using Generalized Advantage Estimation," *ICLR*, 2016. arXiv:1506.02438.

[19] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, "Proximal Policy Optimization Algorithms," arXiv:1707.06347, 2017.

[20] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, S. Yao, "Reflexion: Language Agents with Verbal Reinforcement Learning," *NeurIPS*, 2023. arXiv:2303.11366.

[21] T. Silver, S. Dan, K. Srinivas, J. B. Tenenbaum, L. P. Kaelbling, M. Katz, "Generalized Planning in PDDL Domains with Pretrained Large Language Models," *AAAI*, 2024.

[22] I. Singh et al., "ProgPrompt: Generating Situated Robot Task Plans using Large Language Models," *ICRA*, 2023. arXiv:2209.11302.

[23] G. Wang et al., "Voyager: An Open-Ended Embodied Agent with Large Language Models," arXiv:2305.16291, 2023.

[24] T. Xie et al., "Text2Reward: Reward Shaping with Language Models for Reinforcement Learning," *ICLR*, 2024. arXiv:2309.11489.

[25] S. Yao et al., "ReAct: Synergizing Reasoning and Acting in Language Models," *ICLR*, 2023. arXiv:2210.03629.